Lab 11 — Praktijklab: eerste contact met het cluster

Doel: k3s installeren in je WSL, het cluster verkennen met `kubectl`, een eerste Pod starten, de met Podman gebouwde image `api:1.0` binnenbrengen, en dan met eigen ogen de reconciliatielus zien — via de brug `podman generate kube`.

Vereisten — Docker-deel afgewerkt (labs 00 tot 10). Image `api:1.0` aanwezig in Podman, `~/labo-docker/07/nginx-pod.conf` aanwezig. Ubuntu/WSL met `systemd=true`, `sudo`, en minstens 4 GB RAM voor WSL. De uitvoer hieronder komt van k3s v1.34.1; de naam van de node is die van jouw machine (hier `wsl-ubuntu`).

Meegeleverde bestanden (`files/`) - `pod-api.yaml` — een minimale Pod voor de API van de rode draad.

Stap 0 — k3s installeren (alleen WSL)

```
curl -sfL https://get.k3s.io | sudo sh -s - --write-kubeconfig-mode 644
sudo systemctl status k3s --no-pager | head -3
which kubectl ; ls -l $(which kubectl)
```

Observeer het script dat de binary downloadt, de dienst `k3s.service` aanmaakt en start (`active (running)`), en een `kubectl` die niets anders is dan een link naar `/usr/local/bin/k3s`. Deze stap kan niet herspeeld worden in de sandbox waarin dit lab geschreven is: de uitvoer wordt beschreven, niet aangehaald. Reken op een minuut voor de node klaar is.

Stap 1 — Kennismaken met het cluster

```
kubectl get nodes
kubectl get pods -A
kubectl cluster-info | head -2
ls -l /etc/rancher/k3s/k3s.yaml ; grep -E 'server:' /etc/rancher/k3s/k3s.yaml
kubectl config get-contexts
```

Observeer één node `wsl-ubuntu Ready control-plane … v1.34.1+k3s1`; in `kube-system` de onderdelen die k3s meelevert: `coredns`, `local-path-provisioner`, `metrics-server`, `traefik`, `svclb-traefik-…`, en twee Pods `helm-install-traefik…` in `Completed`; de API server op `https://127.0.0.1:6443`; een kubeconfig in `-rw-r--r--`; en een context `default` zonder namespace.

Uitleg. Eén node speelt control plane en werknode tegelijk. Alles wat je zult zien, loopt via de API server uit `cluster-info`.

```
kubectl api-resources | head -8
kubectl explain pod.spec.containers | head -12
```

Observeer de tabel van objecttypes (`configmaps`, `events`, `namespaces`..., met hun afkortingen en of ze `NAMESPACED` zijn) en de ingebouwde documentatie van het veld `containers` van een Pod.

Stap 2 — Een namespace voor het lab

```
kubectl create namespace lab11
kubectl config set-context --current --namespace=lab11
kubectl config view --minify -o jsonpath='{.contexts[0].context.namespace}' ; echo
```

Observeer `namespace/lab11 created`, `Context "default" modified.`, en dan `lab11`: alle volgende commando's werken in die namespace zonder `-n`.

Stap 3 — Een eerste Pod, zonder bestand

```
kubectl run web --image=docker.io/library/nginx:alpine
kubectl get pods
sleep 8 ; kubectl get pods -o wide
kubectl describe pod web | sed -n '/^Events/,$p'
```

Observeer `pod/web created`, een toestand `ContainerCreating` en dan `Running` met een IP `10.42.0.x` op de node `wsl-ubuntu`, en de chronologie van de Events: `Scheduled` (door `default-scheduler`), `Pulling`, `Pulled … in 3.75s`, `Created`, `Started` (door `kubelet`).

```
kubectl logs web | tail -2
kubectl exec web -- nginx -v
kubectl port-forward pod/web 18080:80 > /dev/null 2>&1 &
sleep 2 ; curl -s localhost:18080 | head -4 ; kill %1
kubectl delete pod web
```

Observeer de logs van nginx, `nginx version: nginx/1.x`, de welkomstpagina via de tunnel `port-forward` (geen gepubliceerde poort: `kubectl` geeft door), en dan `pod "web" deleted from lab11 namespace`.

Uitleg. `logs`, `exec`, `delete` zijn de handelingen van lab 03; `describe` en zijn Events zijn nieuw: het cluster logt elke stap van elk onderdeel.

Stap 4 — De Podman-brug

```
mkdir -p ~/labo-docker/11 && cd ~/labo-docker/11
podman pod create --name demo -p 18090:80
podman run -d --pod demo --name demo-api api:1.0
podman run -d --pod demo --name demo-web -v ~/labo-docker/07/nginx-
pod.conf:/etc/nginx/conf.d/default.conf:ro nginx:alpine
sleep 2 ; curl -s localhost:18090/api/actuator/health ; echo
podman generate kube demo > demo-pod.yaml
grep -E 'kind:|name:|image:|hostPort|hostPath' demo-pod.yaml
podman pod rm -f -t 0 demo
podman kube play demo-pod.yaml | tail -3
sleep 3 ; curl -s localhost:18090/api/actuator/health ; echo
podman kube down demo-pod.yaml | tail -1 ; podman pod ps
```

Observeer `{"status":"UP"}` vanuit de Podman-pod; in de gegenereerde YAML `kind: Pod`, de twee `image:`, een `hostPort: 18090` en een volume `hostPath` naar je nginx-bestand; na `kube play` de pod opnieuw aangemaakt **vanuit de YAML** (`demo-demo-api`, `demo-demo-web`) die weer antwoordt; en daarna geen enkele pod meer na `kube down`.

Uitleg. Podman schrijft en leest de YAML van Kubernetes. Dat bestand beschrijft precies jouw pod — maar `hostPath` en `hostPort` binden het aan je machine; een cluster vervangt ze door een ConfigMap en een Service.

Stap 5 — De ontbrekende image, en dan de import

```
kubectl apply -f <pad-naar-het-lab>/files/pod-api.yaml
sleep 6 ; kubectl get pods
kubectl describe pod api | sed -n '/^Events/,$p' | tail -4
```

Observeer `pod/api created`, dan `ErrImagePull` of `ImagePullBackOff`, met in de Events `Failed to pull image "localhost/api:1.0": … failed to pull and unpack image` en `Back-off pulling image`.

```
podman save localhost/api:1.0 | sudo k3s ctr images import -
sudo k3s crictl images | grep -E 'api|nginx'
sleep 20 ; kubectl get pods -o wide
```

Observeer `Importing … done` aan de importkant; in `crictl images` `localhost/api 1.0` naast `docker.io/library/nginx alpine` (door het cluster gehaald in stap 3); en dan, zonder iets te herstarten, `api 1/1 Running` met zijn IP.

Uitleg. Het cluster heeft zijn eigen imageopslag. De kubelet, die in een lus bleef proberen, vond de image na de import — `imagePullPolicy: IfNotPresent` zegt hem te gebruiken wat aanwezig is.

```
kubectl get pod api -o yaml | grep -E '^  phase:|podIP:|hostIP:'
kubectl exec api -- wget -qO- http://localhost:8080/ ; echo
kubectl port-forward pod/api 18080:8080 > /dev/null 2>&1 &
sleep 2 ; curl -s localhost:18080/actuator/health ; echo ; kill %1
sudo k3s crictl ps | grep -E 'api|CONTAINER' | cut -c1-100
```

Observeer `phase: Running`, een `podIP` in `10.42.0.x` en een `hostIP` (dat van je WSL); `{"message":"bonjour depuis kubernetes","profile":"default"}` (de variabele uit het manifest); `{"status":"UP"}` via de tunnel; en in `crictl ps` een container `api` — die `podman ps` niet toont.

Stap 6 — De reconciliatielus

```
kubectl delete pod api
kubectl get pods
kubectl create deployment api --image=localhost/api:1.0
sleep 6 ; kubectl get deployment,replicaset,pods
```

Observeer de losse Pod voorgoed verwijderd (`No resources found`), en dan drie objecten: `deployment.apps/api 1/1`, `replicaset.apps/api-5f68dd6795 1 1 1`, en een Pod `api-5f68dd6795-xxxxx Running`.

```
POD=$(kubectl get pods -l app=api -o jsonpath='{.items[0].metadata.name}')
kubectl delete pod $POD
sleep 6 ; kubectl get pods
kubectl get deployment api -o jsonpath='{.spec.replicas} {.status.readyReplicas}' ; echo
```

Observeer `pod "api-5f68dd6795-xxxxx" deleted`, en meteen een **nieuwe** Pod `api-5f68dd6795-yyyyy Running`; `1 1`: één replica gewenst, één replica klaar.

Uitleg. Jij hebt de Pod verwijderd; de ReplicaSet heeft het verschil vastgesteld en er een andere aangemaakt, zonder dat je iets vroeg. Dat is het declaratieve model aan het werk — en het onderwerp van lab 12.

Opruimen

```
kubectl delete namespace lab11
kubectl config set-context --current --namespace=default
kubectl get pods -A | grep -c lab11
```

Observeer `namespace "lab11" deleted` (met alles wat erin zat: Deployment, ReplicaSet, Pod), de context terug op `default`, en `0`. De image `localhost/api:1.0` blijft in het cluster; de volgende labs gebruiken ze. Om k3s tussen twee sessies te stoppen: `sudo systemctl stop k3s`; om ooit alles te verwijderen: `/usr/local/bin/k3s-uninstall.sh`.

Wat je nu moet kunnen beweren

- Eén node `Ready`, zes onderdelen in `kube-system`, een API server op `127.0.0.1:6443`, een kubeconfig in `644`.
- `kubectl run` → `Scheduled`, `Pulling`, `Pulled`, `Created`, `Started`; `logs`, `exec`, `port-forward`, `delete` zoals met Podman.
- `podman generate kube` schrijft een `kind: Pod` met `hostPort` en `hostPath`; `podman kube play` speelt hem af.
- Podman-image ≠ clusterimage: `ImagePullBackOff`, dan `podman save | sudo k3s ctr images import -`, en `Running` zonder iets te herstarten.
- Een losse Pod die je verwijdert, komt niet terug; onder een Deployment verschijnt binnen enkele seconden een nieuwe Pod: `1 1`.
- `kubectl delete namespace lab11` neemt alles mee wat het lab aangemaakt heeft.